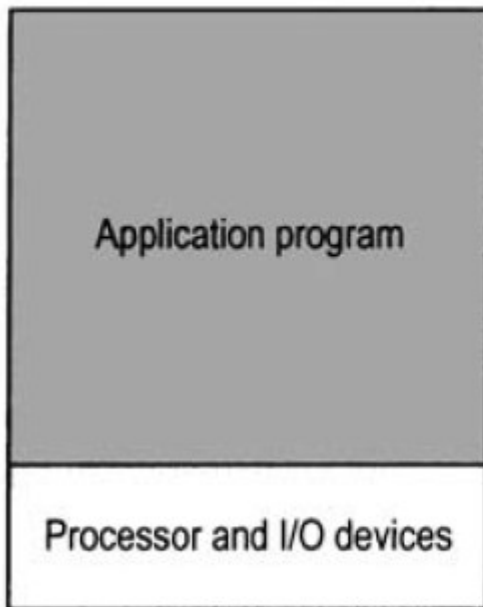
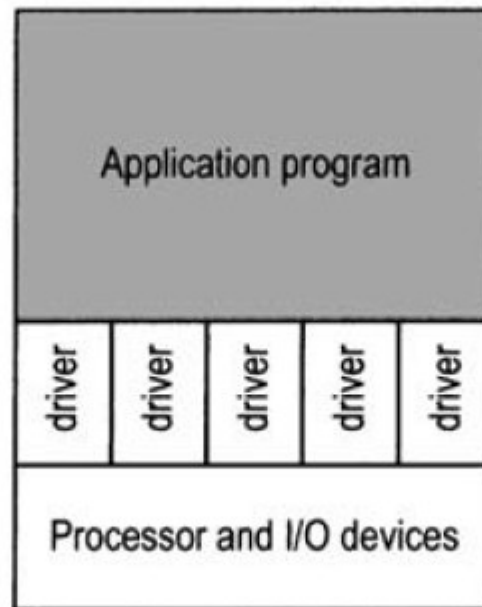


Couche HAL

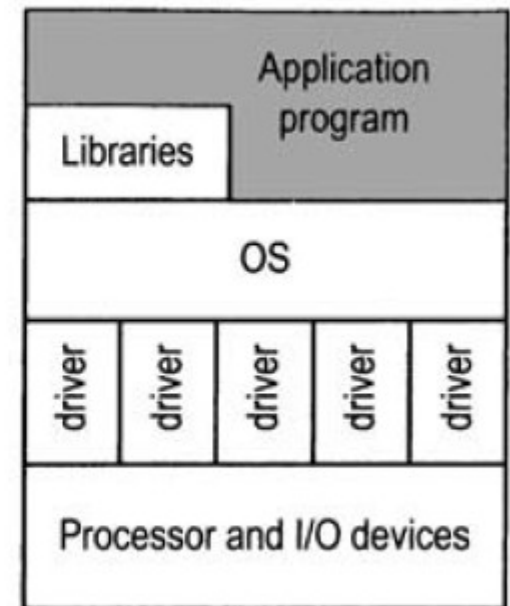
Formes de réalisation d'une application



(a)



(b)



(c)

- Barbone - Bar metal ou FreeDos (Fig 1.a)
 - Ne contient aucun modèle en couches logiciel
 - l'application contrôle le matériel directement
 - l'accès aux périphériques peuvent être développés comme des fonctions
- Couche d'abstraction matérielle (Fig 1.b)
 - Pour permettre la modularité, des pilotes de périphériques peuvent être développés comme des fonctions du programme d'application mis dans une librairie
 - Contrairement aux pilotes sous un OS, le fonctionnement et l'exécution se fait de façon indépendante.
- Desktop like (Fig 1.c)
 - Contient un système d'exploitation à part entière tel que uClinux, petalinux, ...
 - l'Os contrôle et coordonne le matériel à travers les pilotes de périphériques.
 - L'OS est la partie critique de la hiérarchie logicielle
 - Un programme d'application peut seulement accéder au matériel sous-jacent par le biais du système d'exploitation
 - Les pilotes doivent se conformer les exigences imposées par le système d'exploitation.

Processus d'initialisation Bare Metal Like

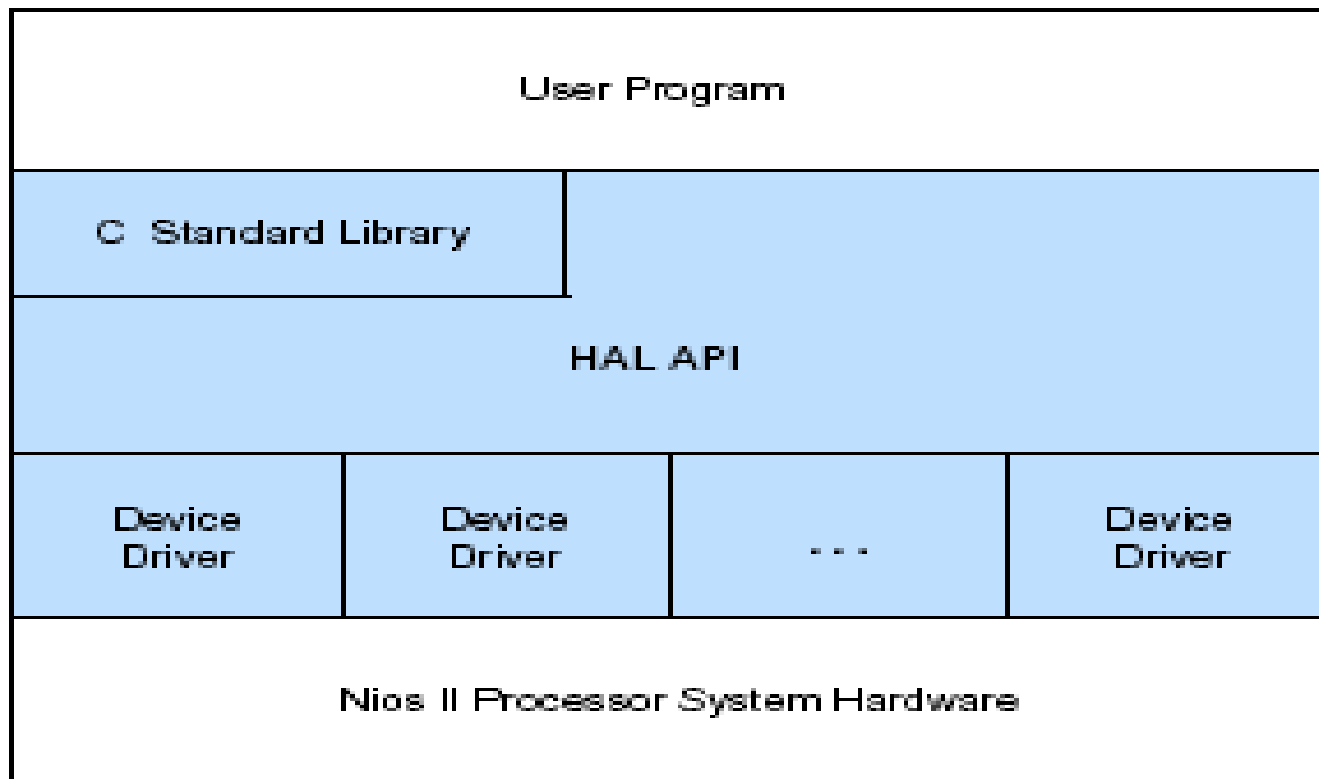
- Un chargeur (boot loader) ajoute généralement un code pour initialiser le processeur.
- Le programme d'application est responsable pour toutes les autres tâches
 - Prépare l'environnement d'exécution par lui-même.
 - Ajoute des codes pour initialiser chaque port E/S
 - Met en place le service de prise en charge des interruptions

Pour afficher un message sur une console, le programme doit mettre en place la fonction I/O correspondante dont l'initialisation du périphérique correspondant

Processus d'initialisation HAL like

- Contrairement à un système d'exploitation, aucun processus HAL explicite n'est présent en mémoire
HAL ne peut donc pas préparer l'environnement dynamique et appeler un programme principal (fonction main).
- HAL fournit un cadre pour collecter automatiquement des informations sur le matériel et fournit les routines pour effectuer le processus d'initialisation.
- Ces routines sont exécutées en premier, et mis en place le temps d'exécution environnement pour le programme d'application.
- Le programme d'application n'a pas besoin d'effectuer toute initialisation explicite

Paradigme HAL



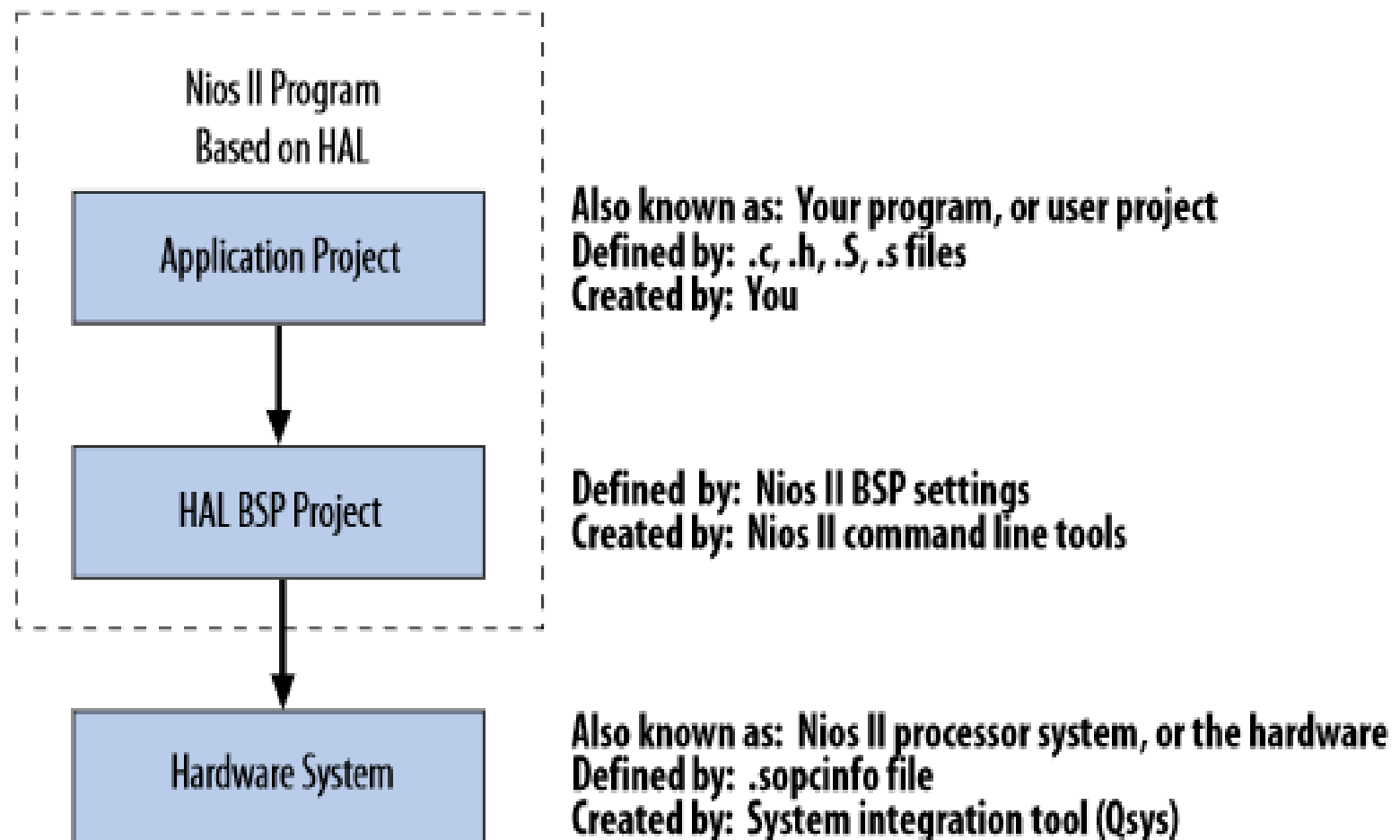
Hardware Abstraction Layer (HAL)

- Isole l'application des modifications matérielles
- A cause de l'abstraction matérielle, l'application est « **indépendante** » **du matériel** grâce aux pilotes qui réalisent l'abstraction matérielle
 - Pilotes (device driver) mode caractère tels que : UART core, JTAG UART, LCD , timer , contrôleur DMA , contrôleur Ethernet MAC/PHY
 - Pilotes mode bloc tel que carte SD, Disque dur,...
- L'API (application program interface) HAL est intégrée à **l'ANSI C standard library**.

Génération des couches HAL

- Génération de la librairie HAL
 1. Qsys génère le système matériel
 2. Nios II IDE ou Monitoring Program génère une couche HAL personnalisée sous forme de librairie adaptée à la configuration matérielle (BSP)
- Un changement dans la configuration matérielle est propagée automatiquement vers la configuration de la couche HAL
- NIOS II utilise l'API à travers une programmation en langage C

The Nios II HAL Project Structure Emphasizing How the HAL BSP Fits In



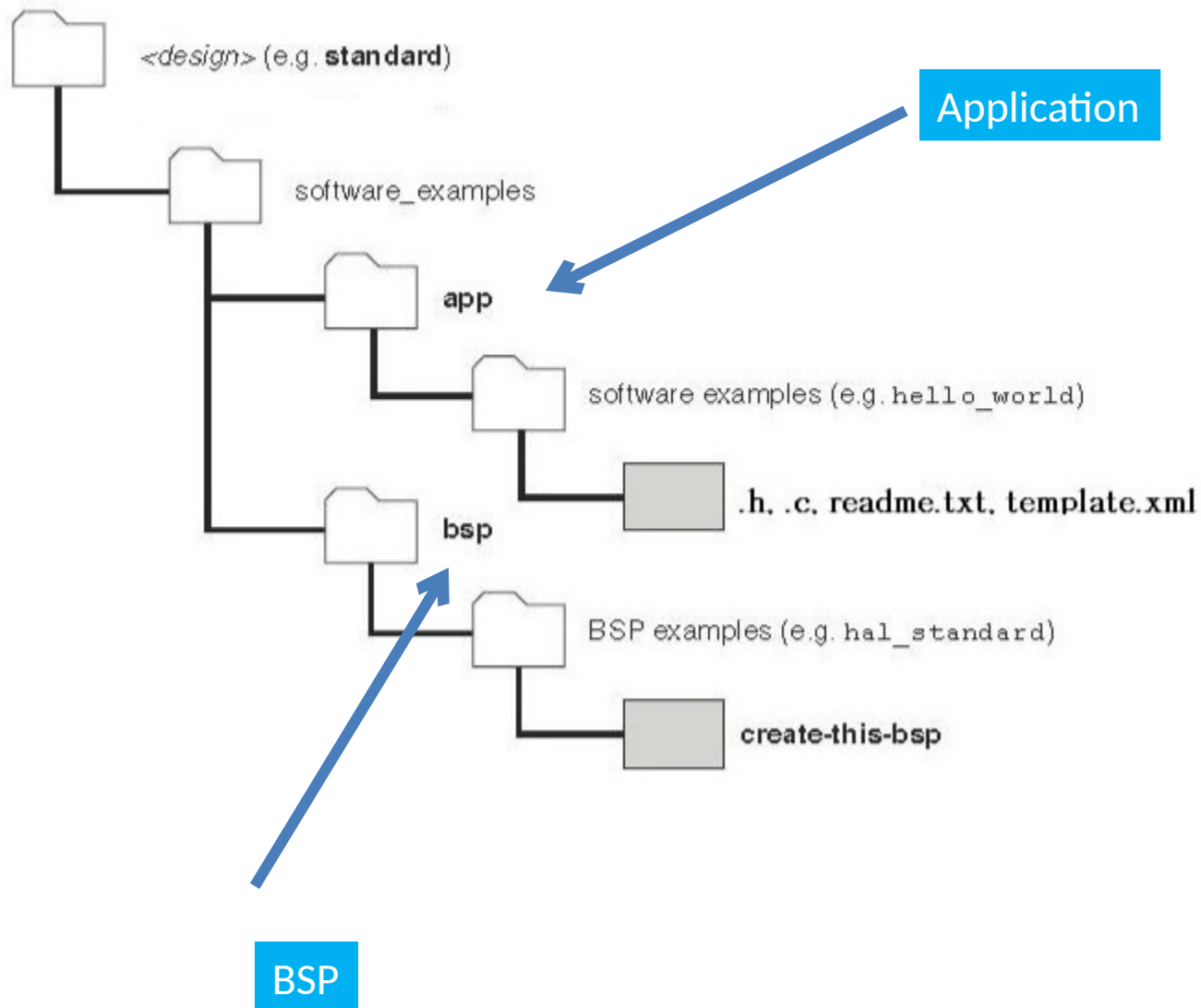
Board Support Package (BSP)

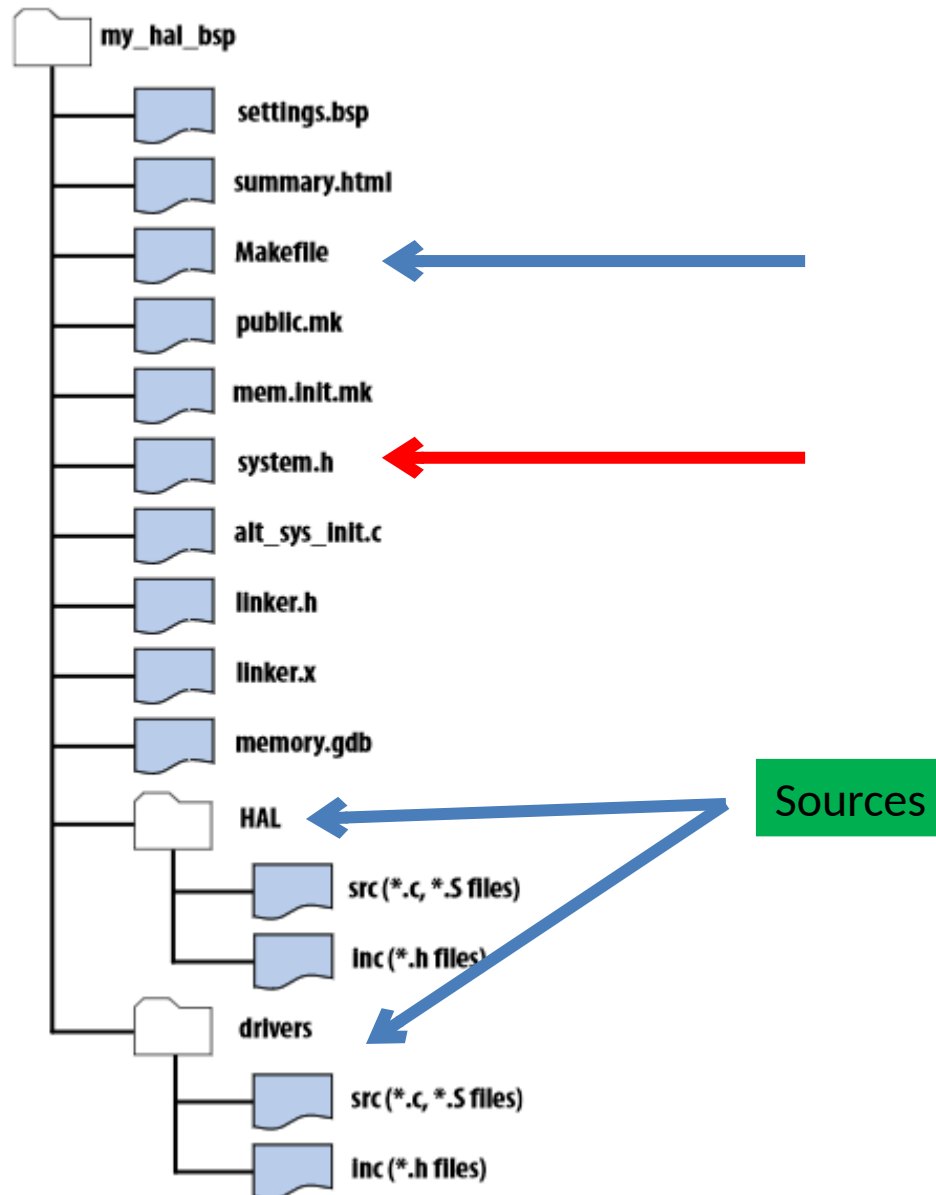
- **Joue un rôle clé** dans le développement de systèmes embarqués.
- **Permet de travailler sur un niveau d'abstraction plus élevé** en fournissant une interface logicielle standardisée pour accéder aux fonctionnalités matérielles spécifiques de la plate-forme.
- **Facilite la portabilité des applications** entre différentes plates-formes matérielles et accélère le développement des applications embarquées en réduisant la complexité liée à la programmation bas niveau du matériel.
- **Fourni par le concepteur du matériel.**

Board Support Package (BSP)

Inclut :

- Drivers
- header files
- device ID number
- system address map





File Name	Function
summary.html	Provides summary documentation of the BSP. You can view summary.html with a hypertext viewer or browser, such as Internet Explorer or Firefox . If you change the settings.bsp file, the SBT updates the summary.html file the next time you regenerate the BSP.
Makefile	Used to build the BSP. The targets you use most often are all and clean . The all target (the default) builds the libhal_bsp.a library file. The clean target removes all files created by a make of the all target.
public.mk	A makefile fragment that provides public information about the BSP. The file is designed to be included in other makefiles that use the BSP, such as application makefiles. The BSP Makefile also includes public.mk .
mem_init.mk	A makefile fragment that defines targets and rules to convert an application executable file to memory initialization files (.dat , .hex , and .flash) for HDL simulation, flash programming, and initializable FPGA memories. The mem_init.mk file is designed to be included by an application makefile. For usage, refer to any application makefile generated when you run the SBT. For more information, refer to the “Creating Memory Initialization Files” chapter.
alt_sys_init.c	Used to initialize device driver instances and software packages.
system.h	Contains the C declarations describing the BSP memory map and other system information needed by software applications.
linker.h	Contains information about the linker memory layout. system.h includes the linker.h file.
linker.x	Contains a linker script for the GNU linker.
memory.gdb	Contains memory region declarations for the GNU debugger.
obj Directory	Contains the object code files for all source files in the BSP. The hierarchy of the BSP source files is preserved in the obj directory.
libhal_bsp.a Library	Contains the HAL BSP library. All object files are combined in the library file. The HAL BSP library file is always named libhal_bsp.a .
settings.bsp	Contains all BSP settings. This file is coded in XML. On the command line, settings.bsp is created by the nios2-bsp-create-settings command, and optionally updated by the nios2-bsp-update-settings command. The nios2-bsp-query-settings command is available to parse information from the settings file for your scripts. The settings.bsp file is an input to nios2-bsp-generate-files . The Nios II SBT for Eclipse provides equivalent functionality.

Programmation du processeur NIOS II

- Programmation UART
 - Standard Input, Standard Output routines en C

```
#include <stdio.h>
#include <string.h>

int main (void)
{
    char* msg = "hello world";
    FILE* fp;
    fp = fopen ("/dev/uart1", "w");
    if (fp)
    {
        fprintf(fp, "%s",msg);
        fclose (fp);
    }
    return 0;
}
```

Mise en œuvre

I- Soit à configurer monitoring programme pour tester un projet avec un BSP généré sous « Monitoring program »

On choisit :

- DE0-Nano (ou DE1) - Basic Computer
 - program type : Program with Device Driver
 - Programme précédent à exécuter
-
- a- expliciter l'objectif de l'application
 - b- expliquer la structure du projet
 - c- expliquer la structure de l'application principale
 - d- tester
 - e- conclusion